

SESSION 1: CLASSICAL DISTRIBUTED TRAINING

*In data parallelism, the **batch size** is the total number of training examples processed together before the model is updated.*

*For example, if you have a total batch size of 64 and 4 devices, each device processes a slice of 16 examples. These individual slices are often called **minibatches**.*

Distributing Training Data Across Devices

Mathematical Foundation:

- In standard SGD with batch size B , the gradient update is:

$$g = \frac{1}{|B|} \sum_{xi \in B} \nabla_{\theta} L(\theta, xi)$$

- With N devices, rank k computes:

$$g_k = \frac{1}{|B_k|} \sum_{xi \in B_k} \nabla_{\theta} L(\theta, xi)$$

- The global gradient is

$$g_{global} = \sum_{k=1}^N \frac{|B_k|}{B} g_k, \quad B = \sum_{k=1}^N |B_k|$$

- For equal local batch sizes:

$$g_{global} = \frac{1}{N} \sum_{k=1}^N g_k$$

Process Overview:

- Split the dataset into non-overlapping subsets
- Perform independent forward/backward passes
- Aggregate gradients using all-reduce
- Apply the same optimizer update on every device

SESSION 2: FEDERATED LEARNING SYSTEMS

What is Federated Learning

- Decentralized training across many edge devices or clients
- Data stays on device; only model updates or gradients are shared

Why it matters

- Privacy preservation and regulatory compliance
- Lower central bandwidth use; clients compute locally

Core challenges

- Clients are intermittent and heterogeneous (CPU/GPU, battery, network)
- Non-IID data across clients; drift and imbalance
- Faults and stragglers; limited communication rounds

FedAvg (Federated Averaging) protocol

- Round t :
 - Server broadcasts current model to selected clients
 - Clients train locally for a few epochs on their data
 - Clients send model deltas or updated weights to server
 - Server aggregates (weighted average by client data size) $\rightarrow$ new global model

Practical tips

- Client sampling: pick a subset each round to scale and reduce stragglers
- Weighting: average by number of local examples to reduce bias
- Compression: quantize/sparsify updates to save bandwidth
- Security/privacy: secure aggregation, differential privacy, device attestation
- Evaluation: track both global and per-client metrics; handle skewed datasets

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)



Non-IID Data Challenges

What's hard

- Client data are skewed and imbalanced; distributions drift over time

Diagnose

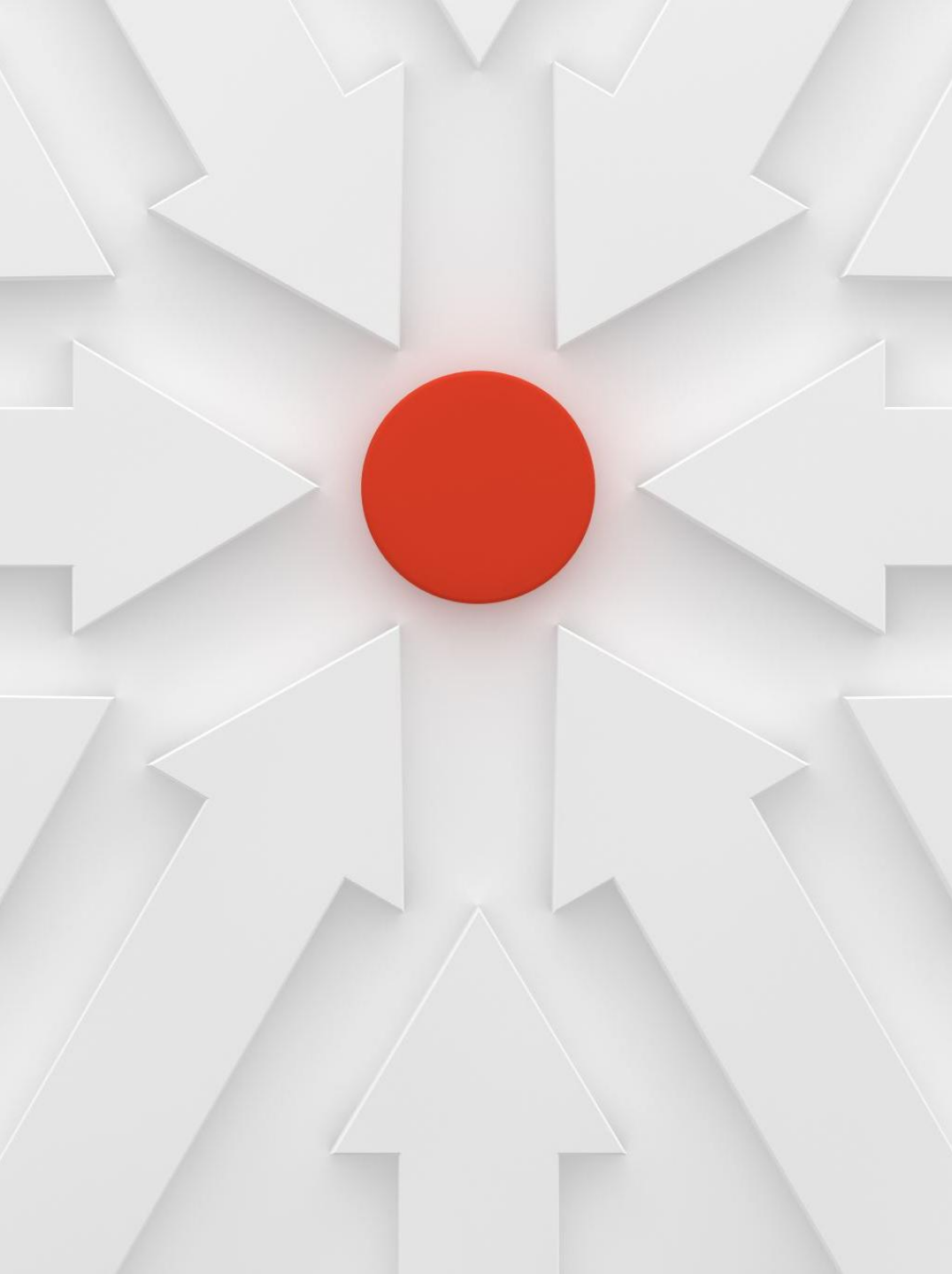
- Compute per-client stats (label/feature histograms, entropy, KS tests)
- Cluster similar clients; track drift across rounds

Adapt aggregation

- Weight by data size, quality, and similarity to global
- Use cohort-wise or personalized aggregation instead of one-size-fits-all

Practical guardrails

- Robustness: clip norms, outlier filtering
- Evaluation: group metrics and worst-case performance



Federated Learning Performance

Convergence factors

- Local steps vs. rounds; data and system heterogeneity; client sampling

Efficiency trade-offs

- More local steps → fewer rounds but risk of drift
- Communication cost vs. accuracy curve

Key metrics

- Rounds to target accuracy; bandwidth per round/total
- Participation rate and straggler impact
- On-device energy/time per round

Performance Optimization

Smart client selection

- Score by data utility, device health, connectivity; diversify to avoid bias

Communication scheduling

- Prefer Wi-Fi + charging; predict availability windows; cache and batch updates

Adaptive training

- Per-device configs: batch size, local steps, precision; auto-tune online
- Early exit for slow/overheating devices

Bandwidth/compute efficiency

- Update compression: sparsify, quantize, delta + server-side momentum
- Partial model updates or adapters; server-side aggregation acceleration

Energy awareness

- Train only on charge or high battery; throttle CPU/GPU; budget per session

Metrics to track

- Time/round, completion rate, bytes/round, on-device energy, accuracy/round

